

A Reconfigurable Photonic Interferometric Computing Framework for Low-Power Matrix Operations and Small-Scale Machine Learning

Santiago Sosa*, Marc DeCarlo†

*Department of Electrical and Computer Engineering, Drexel University, Philadelphia, United States of America
Email: ss5427@drexel.edu

†Department of Electrical and Computer Engineering, Drexel University, Philadelphia, United States of America
Email: mad534@drexel.edu

Abstract—Photonic computing is an emerging paradigm that leverages the high bandwidth and parallelism of optical signals to perform arithmetic operations at low power and high speed. This proposal outlines a 6–12 month project to design, implement, and evaluate a reconfigurable photonic computing framework centered on Mach-Zehnder interferometers (MZIs) or similar optical components. The main contribution is a novel low-level control software stack with real-time calibration algorithms, enabling reliable matrix computations and basic machine learning (ML) tasks on a compact photonic platform. The work spans device-level modeling, algorithmic design, and an end-to-end demonstration of energy-efficient optical matrix operations, potentially providing a pathway toward more scalable optical accelerators.

I. INTRODUCTION

The demand for computational resources continues to surge due to data-intensive applications in artificial intelligence, cryptography, and scientific simulations [?]. Conventional electronic hardware, constrained by the slowing of Moore’s Law and thermal limitations, may not meet the growing need for high-throughput, low-latency operations. Consequently, novel computing paradigms are being investigated to overcome these bottlenecks.

Photonic (or optical) computing has garnered increasing attention for its inherent advantages: low latency, low crosstalk, and extremely high bandwidth [?]. By encoding information in light and using interferometric devices, certain arithmetic operations—especially matrix multiplications—can be performed *in analog* at high speed and with modest power consumption. Various research groups have demonstrated prototype photonic chips capable of implementing small-scale matrix-vector multiplications pertinent to neural networks and signal processing [?], [?].

Despite these achievements, there are two major challenges preventing broader adoption of photonic computing:

- **Reconfigurability and Calibration:** Interferometric networks are sensitive to temperature, fabrication mismatch, and device drift, necessitating active calibration algorithms.

- **Low-Level Programming Models:** There is a pressing need for a robust *software stack* that abstracts the complexities of optical hardware and provides stable, high-level APIs to computational scientists.

The proposed project aims to address these issues via a focus on *reconfigurable photonic interferometric computing* (RPIC) to perform matrix transformations relevant to both HPC and ML tasks. The tight coupling of novel calibration algorithms with domain-specific compilers is expected to yield a reliable platform for small-scale but high-impact demonstrations of optical computing.

II. BACKGROUND AND RELATED WORK

A. Photonic Interferometric Computing

Modern photonic computing architectures often employ a mesh of Mach-Zehnder interferometers (MZIs) to realize *unitary transformations* across multiple optical channels. The seminal work by Reck *et al.* [?] and later improvements by Clements *et al.* [?] demonstrate how reconfigurable arrays of beam splitters and phase shifters can implement arbitrary $N \times N$ linear transformations in an optical network.

Such MZI meshes can be repurposed for *matrix multiplication*, a fundamental operation in HPC and ML (e.g., in neural network layers). However, errors introduced by phase drifts and coupling losses degrade accuracy unless carefully managed. Prior demonstrations have often been point solutions, lacking generalizable calibration frameworks or robust software stacks.

B. Calibration and Control

High-density photonic circuits require continuous or periodic calibration to compensate for:

- **Thermal Drift:** Slight temperature variations change the refractive index and hence the effective optical path length.
- **Fabrication Deviations:** Phase shifters and splitters may exhibit mismatch from their nominal design.

- **Analog Noise:** Real-time fluctuations in voltage drivers and optical sources affect phase accuracy.

Several approaches for calibration range from straightforward iterative scans [?] to more advanced machine learning techniques [?]. However, these are often *ad hoc*, restricted to a specific hardware layout.

C. Low-Level Software for Photonic Hardware

While fields like FPGA-based computing or GPU programming have mature toolchains (e.g., Vivado, CUDA), the photonics domain still lacks widely adopted frameworks for systematically mapping matrix operations onto reconfigurable optical networks. Efforts typically rely on manual parameter tuning or small scripts with limited reusability. A more comprehensive approach would:

- Offer device abstractions for setting phases, reading intensity outputs, and measuring errors.
- Provide a “photonic instruction set” to orchestrate multiple operations (load input, configure MZIs, measure output, etc.).
- Integrate real-time feedback loops for drift compensation.

III. PROJECT OBJECTIVES

This proposal has three interlinked goals, each addressing key constraints of photonic computing:

A. Objective A: Photonic Device Integration & Modeling

- **Model Development:** Construct or adapt a device-level simulator (in Python/Matlab/C++) that captures the transfer functions of MZIs, including phase-voltage relationships, insertion losses, and noise sources.
- **Hardware Setup:** If available, integrate an existing photonic chip featuring a small (e.g., 4×4 or 8×8) MZI mesh into a lab testbed with tunable lasers, photodiodes, and control electronics.

B. Objective B: Control Software and Calibration Algorithms

- **Firmware/Driver Layer:** Implement real-time microcontroller or FPGA-based phase control for each element in the mesh.
- **Calibration Routines:** Develop iterative or ML-based algorithms to converge on the desired matrix transformation while minimizing drift effects.
- **Error-Correction Strategies:** Explore partial digital correction of analog noise, or advanced feedback loops that adapt phases in real time.

C. Objective C: Demonstration of Matrix Computations

- **Matrix Multiplication:** Configure the mesh to multiply a given input vector by a user-defined $N \times N$ matrix (within the scale of the available hardware).
- **Small ML Tasks:** Demonstrate a single-layer inference operation (e.g., a fully connected layer) to showcase an ML use case.
- **Benchmarking:** Measure speed, energy, and accuracy relative to an electronic baseline (CPU or GPU) under a small but realistic dataset.

IV. PROPOSED METHODOLOGY

A. System Architecture

Fig. ?? illustrates the overall architecture. The photonic chip is driven by a digital-to-analog converter (DAC) array for setting phases, while outputs are monitored by a set of photodiodes connected to an analog-to-digital converter (ADC). A microcontroller or FPGA provides real-time feedback. A host computer (e.g., a Python environment) runs higher-level control scripts and compiles matrix operations into device-level commands.

B. Calibration and Configuration

- 1) **Phase Characterization:** Each MZI element undergoes a one-time scan over its full driving voltage range to map voltage to optical phase shift.
- 2) **Iterative Calibration:** To realize a target matrix A , an iterative algorithm (e.g., gradient descent) updates the phases $\{\theta_i\}$ to minimize $\|A_{\text{optical}}(\theta)x - Ax\|$ for a representative set of inputs x .
- 3) **Drift Compensation:** Deploy a real-time feedback loop that continuously monitors outputs for reference inputs and adjusts θ_i to maintain accuracy.

C. Software Stack and Compiler

- 1) **Device-Level API:** A library (in C/C++ or Python) providing functions to set phases, measure outputs, and query calibration states.
- 2) **Matrix Mapping Tool:** A compilation layer that takes user commands (e.g., “multiply vector x by matrix A ”) and translates them into phase settings and readout configurations.
- 3) **Integration with HPC/ML Libraries:** If time permits, develop a rudimentary interface (e.g., a plug-in for NumPy or PyTorch) that offloads matrix operations to the photonic device.

D. Benchmarking Protocol

- **Workloads:** Perform matrix multiplication for dimensions 4×4 or 8×8 , and optionally a small fully connected layer for an ML test (e.g., on random data or a toy dataset).
- **Metrics:** Measure execution latency, throughput (OPS or approximate FLOPS), energy consumption (if a photonic power meter is available), and accuracy (mean squared error or classification accuracy).
- **Comparisons:** Compare with a baseline CPU or embedded GPU for identical tasks and input sizes.

V. PRELIMINARY WORK AND FEASIBILITY

The research group at the University of Example has previously tested a single MZI device for optical beam steering and verified the feasibility of direct microcontroller-based phase control. The group also has access to a 4×4 MZI-based photonic chip from a multi-project wafer (MPW) run. Preliminary

data suggests phase stability of about 5–10 minutes before recalibration is needed. Hence, the hardware integration risk is moderate but manageable.

If the hardware approach encounters delays or if the photonic device is unavailable, a high-fidelity simulator with experimentally validated parameters will be used to emulate the 4×4 or 8×8 mesh in real time. This simulator-based approach still allows rigorous testing of the calibration algorithms and the software stack, though power and speed metrics would be approximate.

VI. PROJECT PLAN AND TIMELINE

The entire project is scoped for 6–12 months to match a short-term grant or advanced undergraduate/graduate research project:

A. Months 1–2: Setup and Low-Level Infrastructure

- Complete literature survey on photonic interferometric networks.
- Build or refine the device-level simulator (Python/Matlab).
- Establish the hardware testbed: configure laser sources, photodiodes, DAC/ADC boards, and microcontroller/FPGA environment.

B. Months 3–4: Calibration and Basic Matrix Multiplication

- Implement iterative calibration and drift compensation routines.
- Demonstrate the ability to realize at least one 4×4 matrix multiplication on the physical chip or simulator.
- Validate accuracy, measure drift, and refine control software.

C. Months 5–6: ML Demonstrations and Benchmarking

- Integrate a simple ML kernel (e.g., a single-layer perceptron) onto the photonic hardware.
- Compare speed, energy (if hardware-based), and accuracy to CPU/GPU benchmarks for small data sets.
- Prepare initial results for a conference submission or internal report.

D. Months 7–12: Extensions (If Available)

- Explore advanced error-correction methods (machine learning-based calibration, partial digital redundancy).
- Scale to larger arrays (e.g., 8×8) if a second-generation chip or design is accessible.
- Publish final results in a photonics or hardware-oriented conference/journal.

VII. EXPECTED CONTRIBUTIONS

- 1) A validated **device-level simulator** or experimental setup that characterizes MZI-based photonic hardware for computing tasks.
- 2) A robust **low-level control software** framework with real-time calibration, suitable for other researchers to adapt.

- 3) Demonstrated ability to perform **basic matrix operations** and a small ML inference task on the photonic chip or simulator, with performance metrics for speed, accuracy, and power.

VIII. CONCLUSION

This project will significantly advance the state-of-the-art in photonic computing by emphasizing the often-neglected aspect of *low-level software control and rigorous calibration methods*. The successful demonstration of reconfigurable matrix operations on a small-scale photonic device (4×4 or 8×8 MZI mesh) will highlight the potential for broader adoption of optical computing in energy-efficient HPC and ML workloads. The short, 6–12 month timeline is feasible given existing hardware availability and prior group experience, making it a compelling opportunity to deliver impactful, publishable results in the rapidly evolving field of photonic computing.

REFERENCES

- [1] Top500 Supercomputer Sites: <https://www.top500.org> (accessed Feb. 2025).
- [2] D. A. B. Miller, “Device requirements for optical interconnects to silicon chips,” *Proc. IEEE*, vol. 97, no. 7, pp. 1166–1185, 2017.
- [3] Y. Shen, N. C. Harris, S. Skirlo *et al.*, “Deep learning with coherent nanophotonic circuits,” *Nature Photonics*, vol. 11, no. 7, pp. 441–446, 2017.
- [4] N. C. Harris, Y. Zhou, M. Prabhu *et al.*, “Linear programmable nanophotonic processors,” *Optica*, vol. 5, no. 12, pp. 1623–1631, 2018.
- [5] M. Reck, A. Zeilinger, H. J. Bernstein, and P. Bertani, “Experimental realization of any discrete unitary operator,” *Phys. Rev. Lett.*, vol. 73, no. 1, p. 58, 1994.
- [6] W. R. Clements, P. C. Humphreys, B. J. Metcalf *et al.*, “An optimal design for universal multiport interferometers,” *Optica*, vol. 3, no. 12, pp. 1460–1465, 2016.